■■ SAFAR: Physical Automotive Hardware Deployment Guide

Direct Transition from Simulation to Real-World Embedded Vehicle Platform

1. Architectural Continuity: Why 90%+ of Code is Reused

SAFAR was architected from Day 1 with strict protocol decoupling (defined in interfaces/protocols.md). Because perception, kinematic physics reasoning, and actuator overrides operate across standardized network contracts, deploying to physical automotive hardware requires **zero changes to the core algorithms**. You only swap the virtual simulation bridges (TCP 9001 / UDP 9003) with real physical hardware device drivers (OpenCV V4L2 cameras and SocketCAN bus).

2. Hardware Topology & Data Pipeline

3. 1-to-1 Mapping: Simulation vs. Physical Hardware

Subsystem Layer	Simulation (UE5 / PC)	Physical Vehicle Hardware	Interface & Implementation
Vision Sensors	USceneCaptureComponent2D (TextureRenderTarget2D)	Stereo Cameras (GMSL2 / USB 3.0, ZED 2i / Sony IMX327)	PhysicalCameraSource (OpenCV V4L2 / GStreamer)
Vehicle IMU & Speed	Chaos Vehicle Movement Velocity	Vehicle CAN-Bus (Wheel Speed) + MPU-6050 6-DoF IMU	SocketCAN / OBD-II Diagnostic Pulse Reader
AI Perception	PyTorch YOLOv8 on PC GPU	NVIDIA Jetson Orin (TensorRT FP16/INT8 Engine)	8.5ms latency @ 120 FPS (< 15W power draw)
Safety Core	C++ safar_core on Windows/Linux	safar_core running as high-priority Linux RT service	< 4.5ms deterministic physics evaluation
Braking Actuation	VehicleMovement->SetBrakeInput()	Bosch iBooster / Electronic Brake Booster (EBB)	Automotive CAN Frame commanding target deceleration
Reverse Protection	VehicleMovement->SetHandbrakeInput()	Electronic Parking Brake (EPB) Solenoid Relay	CAN lock command when speed <= 0.5 m/s

4. Physical Hardware Subsystem Breakdown

- **Camera Ingestion (Dual Stereo Pair)**: Connected via MIPI CSI-2 or GMSL2 serialized cables. safar_perception/sensor_interface.py directly activates the physical camera with PhysicalCameraSource(camera_index=0).
- **NVIDIA Jetson Orin Edge Compute**: Compiles yolo11n.pt into a TensorRT execution engine (yolo11n.engine). Runs object detection, Indian auto-rickshaw remapping, rider-bike fusion, and pothole classification at 120 FPS on 12V automotive power.
- **CAN-Bus Actuator Bridge (Bosch iBooster)**: Modern road vehicles utilize electromechanical brake boosters (e.g. Bosch iBooster). SAFAR broadcasts CAN frames at 100 Hz specifying target deceleration (e.g. 6.5 m/s² nominal, 8.5 m/s² emergency) to pressurize the hydraulic lines.
- **Anti-Roll Reverse Lock**: When the vehicle reaches a complete stop ($v \le 0.5\text{ m/s}$), SAFAR commands the Electronic Parking Brake (EPB) over CAN to prevent hill roll-back or automatic transmission creep.

5. Prototype Bill of Materials (BOM)

Item Component	Recommended Hardware	Role in Physical SAFAR Pipeline	Approx. Cost
AI Edge Compute	NVIDIA Jetson Orin Nano (8GB)	Runs YOLO TensorRT + Pothole ML + safar_core C++17	~$249
Vision Sensors	ZED 2i / Dual Sony IMX327 (GMSL2)	Captures 1080p stereo frames & optical ground plane	~$199 – $449
CAN Transceiver	Waveshare 2-CH CAN FD HAT / CANable	Interfaces Jetson to vehicle CAN-Bus for speed & braking	~$25
Vehicle Telemetry	OBD-II Cable / 6-DoF MPU-6050 IMU	Supplies raw wheel speed pulses, yaw rate & pitch	~$15
Power Regulation	12V to 19V/5V Automotive DC-DC	Clean isolated power delivery from car 12V battery	~$20
Cockpit HUD	5-inch HDMI / DSI Touchscreen OLED	Renders real-time ADAS threat gauge & speed alerts	~$45

6. Automotive Safety & Failsafe Mechanisms

- **Hardware Watchdog Protection (250ms)**: If camera communication or AI inference stalls for $> 250\text{ms}$, safar_core automatically disengages autonomous braking and returns 100% control to the driver.
- **Physical Driver Authority**: The physical driver brake pedal mechanically overrides the electronic booster at all times.
- **Hardware Emergency Stop Relay**: A dashboard kill switch disconnects power to the CAN transceiver, isolating the vehicle immediately.